ServiceRegistry

System Design Description

Abstract
This document describes the design and implementation of the ServiceRegistry core system of Arrowhead (documentation edition 5.3.0) in Java Spring (`ah5-core-java-spring/serviceregistry`).

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**

3 Implementation **4**
3.1 Implementation language and tools 4
3.2 Functional properties implementation 4
3.3 Non functional properties implementation 5
3.4 Deployment 5
3.5 Local modifications 6
3.6 Verification 6

4 Services **7**

5 Go and Rust implementation variants **7**
5.1 Go implementation 7
5.2 Rust implementation 7

6 References **8**

7 Revision History **10**
7.1 Amendments 10
7.2 Quality Assurance 10

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the ServiceRegistry system implementation, which realizes the ServiceRegistry SysD [1]. The ServiceRegistry is the first core system of an Arrowhead local cloud. It keeps the authoritative register of the devices, systems and service instances of the cloud and of the interface templates that describe how services are reached. Every other system registers itself and its services there and finds the core services through it; orchestrators use it to discover candidate providers. In Section 3, we describe implementation details of the system.

3 Implementation

This implementation is based on the SysD document ServiceRegistry [1], the upstream Eclipse Arrowhead Java Spring implementation 5.2.2 [2] and the local changes listed in Section 3.5.

3.1 Implementation language and tools

- Java 21, Spring Boot 3.4.9 (web, security, data JPA, Quartz), springdoc-openapi 2.8 (Swagger UI), Log4j 2.
- Maven module `serviceregistry` of `ah5-core-java-spring`; it depends on `arrowhead-common-utils` and `arrowhead-data-transfer-objects` 5.2.2 from the local `ah5-common-java-spring` (Eclipse Paho MQTT v3 1.2.5, RabbitMQ AMQP client 5.22.0).
- Database: MariaDB/MySQL (MariaDB Connector/J; MySQL 8.4 server in the test cloud), schema validated by Hibernate (`ddl-auto=validate`); SQL scripts in `src/main/resources/database`.
- Stateless request processing; all state is in the database. Scheduled jobs use Quartz.

3.2 Functional properties implementation

The Java packages below `eu.arrowhead` implement the system:

Package	Responsibility
`api.http, api.mqtt`	REST controllers and MQTT topic handlers of the five services; the lookup filter that marks restricted lookups.
`service`	Device, system and service discovery and management logic; `ReservedServiceDefinitions` (J9); discovery and interface policies.
`service.validation, service.normalization`	Input validation and name normalization; interface validators per template property.
`jpa`	Entities and repositories of the register; database services with bulk operations.
`init`	Registration of the system's own services at start-up, extra interfaces and interface policy override (local).
`collector`	`DatabaseCollectorDriver`: the ServiceRegistry resolves services from its own database.

Every request passes the common filter chain: authentication filter (by `authentication.policy`), optional blacklist filter, management service filter (for management services), then the controller, validation and normalization, the service logic and the database service. The same logic is reachable through the MQTT topic handlers.

3.2.1 Data handled and stored

Database `ah_serviceregistry`:

Tables	Content
`device, device_address`	Devices and their addresses.
`system_, system_address, device_system_connector`	Systems, addresses and device assignment.
`service_definition`	Service definition names.
`service_instance, service_instance_interface`	Service instances and their interfaces (properties as JSON).

Tables	Content
`service_interface_template`, `service_interface_template_property`	Interface templates and property rules; default inserts add `generic_http(s)`, `generic_mqtt(s)` and, locally, `generic_amqp(s)`.
`logs`	Log entries for `get-log`.

The data models of the interfaces are the Java records of `eu.arrowhead.dto` (common library); the IDD documents name them without the `DTO` suffix. The records used by this system are: `AddressDTO`, `DeviceListRequestDTO`, `DeviceListResponseDTO`, `DeviceLookupRequestDTO`, `DeviceQueryRequestDTO`, `DeviceRequestDTO`, `DeviceResponseDTO`, `KeyValuesDTO`, `LogEntryDTO`, `LogEntryListResponseDTO`, `LogRequestDTO`, `MetadataRequirementDTO`, `PageDTO`, `ServiceDefinitionListRequestDTO`, `ServiceDefinitionListResponseDTO`, `ServiceDefinitionResponseDTO`, `ServiceInstanceCreateListRequestDTO`, `ServiceInstanceCreateRequestDTO`, `ServiceInstanceInterfaceRequestDTO`, `ServiceInstanceInterfaceResponseDTO`, `ServiceInstanceListResponseDTO`, `ServiceInstanceLookupRequestDTO`, `ServiceInstanceQueryRequestDTO`, `ServiceInstanceRequestDTO`, `ServiceInstanceResponseDTO`, `ServiceInstanceUpdateListRequestDTO`, `ServiceInstanceUpdateRequestDTO`, `ServiceInterfaceTemplateListRequestDTO`, `ServiceInterfaceTemplateListResponseDTO`, `ServiceInterfaceTemplatePropertyDTO`, `ServiceInterfaceTemplateQueryRequestDTO`, `ServiceInterfaceTemplateRequestDTO`, `ServiceInterfaceTemplateResponseDTO`, `SystemListRequestDTO`, `SystemListResponseDTO`, `SystemLookupRequestDTO`, `SystemQueryRequestDTO`, `SystemRegisterRequestDTO`, `SystemRequestDTO`, `SystemResponseDTO`.

3.3 Non functional properties implementation

3.3.1 Security

TLS and client authentication are configured with the Spring `server.ssl.*` properties (PKCS#12 keystore and truststore). The authentication filters of the common library implement the four authentication policies; `TokenCertificateBinding` binds identity tokens to the client certificate and `CrlRevocationConfig` enables CRL checking in Tomcat (both local). The private key of the keystore is also used to sign the system's outgoing requests where required.

3.3.2 Power management

None; the system is a server process without power states. The `sysMonitor` `power` observation always reports line supply.

3.3.3 Internal monitoring

Log4j 2 logs to stdout and rolling files; log entries are also written to the `logs` table and served by `generalManagement.get-log`. The native `sysMonitor` publisher (`eu.arrowhead.common.sysmonitor`) publishes `echo`, `power` and `status` every 30 s.

3.3.4 Configuration

Configuration is read from `application.properties` and environment variables (Spring relaxed binding, e.g. `SERVICE_DISCOVERY_POLICY`); secrets come from files through `SPRING_CONFIG_IMPORT=configtree:/run/secrets/`. The properties are listed in the SysD. `generalManagement.get-config` returns the public ones.

3.4 Deployment

The test cloud (`Arrowhead-test-cloud-java`) builds the image `arrowhead-serviceregistry:5.2.2-sysmonitor` from the released image `aitiaiiot/arrowhead-serviceregistry:5.2.2` with the locally

built `arrowhead-serviceregistry-5.2.2.jar` (`java-patch/Dockerfile`). The container runs read-only with the configuration baked into `/app/config`, only its keystores mounted, and publishes port 8443 on the loopback address of the host. Its database runs in a separate container.

3.5 Local modifications

Compared to upstream 5.2.2, this workspace changes:

- Startup is refused when the effective configuration is below the secure profile (`SecurityProfileGuard`).
- Reserved service definitions for the core and support systems (finding J9), configurable with `service.definition.reserved`.
- The own interfaces are registered with `CERT_AUTH` behind TLS and with `NONE` for declared identities or without TLS; `service.interface.policy` overrides this; `service.registry.extra.interfaces` registers additional ingress interfaces.
- Native `sysMonitor` publisher (common library), always registered with `CERT_AUTH`; interface templates `generic_amqp` and `generic_amqps` in the default inserts.
- Token-to-certificate binding of identity tokens and CRL checking of client certificates (common library).
- MariaDB Connector/J instead of MySQL Connector/J in the default configuration.

3.6 Verification

The module has 31 JUnit test classes; the regression report set recorded on 2026-10-20 covers 699 tests without failures. Live-cloud smoke tests are a separate verification step; the plant-management migration is not claimed as deployed by this update.

4 Services

The implementation of the services is based on the SD and IDD documents referenced below.

Services produced	**SysD ref**	**SD ref**	**IDD ref**
`deviceDiscovery`	[1]	[3]	[4], [5]
`systemDiscovery`	[1]	[6]	[7], [8]
`serviceDiscovery`	[1]	[9]	[10], [11]
`serviceRegistryManagement`	[1]	[12]	[13], [14]
`generalManagement`	[1]	[15]	[16], [17]
`sysMonitor`	[1]	[18]	[19]
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
identity	[20]	[21]	–
authorization	[22]	[23]	–
blacklistDiscovery	Blacklist	–	–
sysMonitor (publisher)	AMQP broker (SysMonitor)	–	–

5 Go and Rust implementation variants

5.1 Go implementation

Primary system folder: `Eclipse-Arrowhead-go/ah5-core-go/serviceregistry`.

The system owns its Go module, launcher and build configuration. Use the per-system `Makefile` and `go.mod`; the containing Go tree is the build source, not a historical Arrowhead copy. Ordinary native roles use independent identities and JSON snapshot state rather than the Java database. Do not assume Java SQL migration, clustering or identical transaction/retry semantics. The registry has its own configuration and optional native AMQP monitor; other roles must be checked against their own registrations.

Listening port. Default 8443 (server.port in application.properties). Override with server.port, SERVER_PORT=N or -server.port N; the host is server.address. The native registry API, outsourced authentication, JSON persistence, optional AMQP telemetry and differences from Java are specified in `Eclipse-Arrowhead-go/ah5-core-go/serviceregistry/UPSTREAM-README.md`. It has no MQTT application API according to that profile. Native design/route evidence: `Eclipse-Arrowhead-go/ah5-core-go/serviceregistry/documentation`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

5.2 Rust implementation

Primary system folder: `Eclipse-Arrowhead-rust/ah5-core-rust/serviceregistry`.

The per-system entry point is built by the `arrowhead-core-rust` crate. Build from the primary Rust workspace with the applicable package/binary selection. Documentation links can resolve into the shared core/support catalogue. Independent process identity and state remain required.

The ordinary native profile uses Tokio/Axum and rustls with peer-identity checks and independent locked JSON state. Network effects are not distributed database transactions. The precise trust, CRL reload and persistence guarantees are those of the native profile, not the Java implementation.

Native design/route evidence: `Eclipse-Arrowhead-rust/ah5-core-rust/documentation/ServiceRegistry`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

6 References

[1] J. Delsing, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, serviceregistry/documentation/v5.3.0/ServiceRegistry-v5.3.0-SysD.pdf.

[2] AITIA International, “Eclipse arrowhead 5 core systems (java spring) and documentation,” https://aitia-iiot.github.io/ah5-docs-java-spring/, 2025.

[3] J. Delsing, “deviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.3.0, 2026, serviceRegistry-v5.3.0-deviceDiscovery-SD.pdf.

[4] ——, “deviceDiscovery HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.3.0, 2026, serviceRegistry-v5.3.0-deviceDiscovery-HTTP-IDD.pdf.

[5] ——, “deviceDiscovery MQTT/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.3.0, 2026, serviceRegistry-v5.3.0-deviceDiscovery-MQTT-IDD.pdf.

[6] ——, “systemDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.3.0, 2026, serviceRegistry-v5.3.0-systemDiscovery-SD.pdf.

[7] ——, “systemDiscovery HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.3.0, 2026, serviceRegistry-v5.3.0-systemDiscovery-HTTP-IDD.pdf.

[8] ——, “systemDiscovery MQTT/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.3.0, 2026, serviceRegistry-v5.3.0-systemDiscovery-MQTT-IDD.pdf.

[9] ——, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.3.0, 2026, serviceRegistry-v5.3.0-serviceDiscovery-SD.pdf.

[10] ——, “serviceDiscovery HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.3.0, 2026, serviceRegistry-v5.3.0-serviceDiscovery-HTTP-IDD.pdf.

[11] ——, “serviceDiscovery MQTT/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.3.0, 2026, serviceRegistry-v5.3.0-serviceDiscovery-MQTT-IDD.pdf.

[12] ——, “serviceRegistryManagement – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.3.0, 2026, serviceRegistry-v5.3.0-serviceRegistryManagement-SD.pdf.

[13] ——, “serviceRegistryManagement HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.3.0, 2026, serviceRegistry-v5.3.0-serviceRegistryManagement-HTTP-IDD.pdf.

[14] ——, “serviceRegistryManagement MQTT/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.3.0, 2026, serviceRegistry-v5.3.0-serviceRegistryManagement-MQTT-IDD.pdf.

[15] ——, “generalManagement – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.3.0, 2026, serviceRegistry-v5.3.0-generalManagement-SD.pdf.

[16] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.3.0, 2026, serviceRegistry-v5.3.0-generalManagement-HTTP-IDD.pdf.

[17] ——, “generalManagement MQTT/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.3.0, 2026, serviceRegistry-v5.3.0-generalManagement-MQTT-IDD.pdf.

[18] ——, “sysMonitor – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.3.0, 2026, serviceRegistry-v5.3.0-sysMonitor-SD.pdf.

[19] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.3.0, 2026, serviceRegistry-v5.3.0-sysMonitor-AMQP-IDD.pdf.

[20] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, authentication/documentation/v5.3.0/Authentication-v5.3.0-SysD.pdf.

[21] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.3.0, 2026, authentication-v5.3.0-identity-SD.pdf.

[22] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, consumer-authorization/documentation/v5.3.0/ConsumerAuthorization-v5.3.0-SysD.pdf.

[23] ——, “authorization – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.3.0, 2026, consumerAuthorization-v5.3.0-authorization-SD.pdf.

7 Revision History

7.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-11-07	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

7.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	